

NANDA Implementation PRD v0.1

Internet of AI Agents Index - Build Plan for Codex and Implementation Team

Prepared for architecture review and Sprint Zero setup

1. Objective

Build a working v0 implementation of NANDA: a DNS-like registry and resolver for AI agents that supports compact signed agent anchors, signed AgentFacts metadata, trust verification, dynamic endpoint selection, revocation checks, auditability, and observability.

The v0 goal is not trillion-agent scale. The goal is to prove the core protocol and architecture end-to-end in a local/developer environment with production-shaped boundaries.

2. v0 Scope

2.1 In scope

1. Register an agent with a stable AgentID.
2. Create and store a compact signed AgentAddr120 record.
3. Publish AgentFacts as JSON.
4. Resolve an agent through the NANDA resolver.
5. Verify AgentAddr120 signature, AgentFacts digest/commitment, capability credential issuer allowlist, credential status/revocation state, and endpoint binding.
6. Select a valid endpoint using a simple routing policy.
7. Emit audit events for every important state transition.
8. Emit OpenTelemetry-style traces, metrics, and logs.
9. Provide Docker Compose deployment for local testing.
10. Provide automated tests across unit, integration, contract, security, audit, and load categories.

2.2 Out of scope for v0

11. Public global federation.
12. Real OHTTP implementation.
13. Real IPFS/Tor private facts distribution.
14. Full W3C VC production-grade interoperability.
15. Envoy xDS production routing.
16. Trillian-backed transparency log.
17. ZK proofs.
18. Behavioral trust/reputation scores.

For v0, these are represented by clean interfaces and mockable adapters.

3. System Components

3.1 Resolver API

Primary service used by agents and applications. It accepts resolution requests, looks up L1 AgentAddr120, verifies the L1 signature, fetches L2 AgentFacts, verifies the AgentFacts commitment, verifies capability credentials, checks revocation state, evaluates routing policy, returns selected endpoint and proof bundle, and emits audit events and telemetry. Recommended language: Go.

3.2 Registration API

Administrative service used to register or update agents. It accepts agent registration requests, normalizes AgentID, stores AgentFacts, creates compact AgentAddr120, signs AgentAddr120, stores the L1 anchor record, and emits registration audit events. For v0, this may live in the same Go binary as the Resolver API, but should be separated internally by package boundaries.

3.3 L1 Lean Index Store

Stores compact anchor records. v0 technology: PostgreSQL authoritative store and optional Redis hot cache.

3.4 AgentFacts Store

Stores rich agent metadata. v0 technology: local filesystem adapter for tests, with optional S3-compatible adapter later.

3.5 Trust Verifier

Verifies capability claims and issuer authority. v0 simplification: JSON credential objects, Ed25519 signatures or JCS-style canonical hash for internal consistency, local issuer allowlist, and local revocation list/status map.

3.6 Revocation Service

Maintains credential status. v0 technology: PostgreSQL or in-memory map for test mode. If a credential is revoked, resolution must fail. If credential status is stale beyond configured freshness, high-risk resolution must fail closed.

3.7 Routing Policy Engine

Selects endpoint from verified AgentFacts. v0 can implement policy in Go with an interface that allows OPA/Rego later.

3.8 Audit Logger

Every important operation must emit an audit event. v0 technology: PostgreSQL append-only audit table with hash chaining.

3.9 Observability

v0 must expose /healthz, /readyz, /metrics, structured JSON logs, and trace IDs in request/response logs.

3.3.1 L1 database tables

```
CREATE TABLE agent_addr_records (  
  agent_hash BYTEA PRIMARY KEY,
```

```

agent_id TEXT NOT NULL,
record BYTEA NOT NULL,
sequence BIGINT NOT NULL,
ttl_seconds INT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE TABLE agent_addr_history (
  id BIGSERIAL PRIMARY KEY,
  agent_hash BYTEA NOT NULL,
  agent_id TEXT NOT NULL,
  record BYTEA NOT NULL,
  sequence BIGINT NOT NULL,
  event_type TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

3.4.1 FactsStore interface

```

type FactsStore interface {
  Put(ctx context.Context, agentID string, facts []byte) (FactsPointer, error)
  Get(ctx context.Context, pointer FactsPointer) ([]byte, error)
}

```

3.5.1 TrustVerifier interface

```

type TrustVerifier interface {
  VerifyAgentFacts(ctx context.Context, facts AgentFacts, requiredCapability string, policy
TrustPolicy) (TrustDecision, error)
}

```

3.8.1 Audit table

```

CREATE TABLE audit_events (
  id BIGSERIAL PRIMARY KEY,
  event_id UUID NOT NULL UNIQUE,
  event_type TEXT NOT NULL,
  agent_id TEXT,
  agent_hash TEXT,
  actor_hash TEXT,
  decision TEXT,
  reason TEXT,
  event_json JSONB NOT NULL,
  previous_hash TEXT,
  event_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- event_hash = SHA256(previous_hash || canonical_event_json)

```

3.9.1 Required metrics

- nanda_resolve_latency_ms
- nanda_l1_lookup_latency_ms
- nanda_facts_fetch_latency_ms
- nanda_trust_verify_latency_ms
- nanda_resolution_total
- nanda_resolution_denied_total
- nanda_signature_verify_failed_total
- nanda_facts_digest_mismatch_total

- nanda_credential_revoked_total
- nanda_audit_write_failed_total

4. Core Data Models

4.1 AgentAddr120

Binary size must be exactly 120 bytes.

payload[56]		
version	uint8	1 byte
flags	uint8	1 byte
ttl_seconds	uint16	2 bytes
sequence	uint32	4 bytes
agent_id_hash128	bytes16	16 bytes
facts_ptr_hash128	bytes16	16 bytes
credential_set128	bytes16	16 bytes
signature	bytes64	64 bytes

4.1.1 Validation rules

- Reject if byte length is not 120.
- Reject unsupported version.
- Reject zero TTL.
- Reject sequence lower than current sequence.
- Reject invalid Ed25519 signature.
- Reject hash mismatch with fetched AgentFacts.

4.2 AgentFacts

Required fields for the v0 AgentFacts document:

```
{
  "id": "nanda:enterprise.acme/procurement/negotiator-v2",
  "issuer": "did:web:acme.com:compliance",
  "validFrom": "2026-05-27T00:00:00Z",
  "validUntil": "2026-06-27T00:00:00Z",
  "agent": {
    "controller": "did:web:acme.com",
    "softwareDigest": "sha256:...",
    "capabilities": ["supply-chain.price-negotiation"]
  },
  "factsAccess": {
    "primaryFactsURL": "file://facts/acme-negotiator-v2.json"
  },
  "endpoints": [
    {
      "type": "adaptiveResolver",
      "url": "https://resolver.agent.acme.com/v1/dispatch",
      "ttlSeconds": 30,
      "endpointBindingHash": "sha256:..."
    }
  ],
  "credentials": [
    {
      "id": "cred-001",
      "type": "AgentCapabilityCredential",
      "issuer": "did:web:acme.com:compliance",

```

```

        "capability": "supply-chain.price-negotiation",
        "status": "valid",
        "signature": "..."
    }
}

```

5. API Requirements

5.1 Register Agent

POST /v1/agents/register

Request:

```

{
  "agentId": "nanda:enterprise.acme/procurement/negotiator-v2",
  "facts": { }
}

```

Response:

```

{
  "agentId": "nanda:enterprise.acme/procurement/negotiator-v2",
  "agentHash": "...",
  "agentAddrRecordBase64": "...",
  "factsPointer": "...",
  "sequence": 1
}

```

Acceptance criteria:

- Creates AgentFacts.
- Creates exactly 120-byte AgentAddr120.
- Stores L1 record.
- Emits agent.registered audit event.
- Resolver can resolve the agent after registration.

5.2 Resolve Agent

POST /v1/resolve

Request:

```

{
  "agentId": "nanda:enterprise.acme/procurement/negotiator-v2",
  "requiredCapability": "supply-chain.price-negotiation",
  "privacyMode": "direct",
  "trustPolicy": "enterprise-strict"
}

```

Response:

```

{
  "agentId": "nanda:enterprise.acme/procurement/negotiator-v2",
  "endpoint": "https://resolver.agent.acme.com/v1/dispatch",
  "endpointType": "adaptiveResolver",
  "ttlSeconds": 30,
  "trustDecision": "allowed",
  "proofBundle": {

```

```

    "agentAddrSignatureVerified": true,
    "agentFactsDigestVerified": true,
    "credentialChainVerified": true,
    "credentialStatus": "valid"
  }
}

```

Acceptance criteria:

- Valid agent resolves successfully.
- Invalid signature fails.
- Missing capability fails.
- Untrusted issuer fails.
- Revoked credential fails.
- Facts digest mismatch fails.
- Audit event is written for allow and deny decisions.

5.3 Revoke Credential

POST /v1/credentials/{credential_id}/revoke

Acceptance criteria:

- Credential becomes revoked.
- Future resolution requests fail.
- Revocation audit event is written.
- Resolver failure occurs within configured revocation freshness window.

5.4 Audit Events

GET /v1/audit/events?agentId=...

Acceptance criteria:

- Returns ordered events.
- Includes event hash and previous hash.
- Tamper verification endpoint can validate hash chain.

6. Repository Structure

```

nanda/
├── cmd/
│   └── nanda-server/
│       └── main.go
├── internal/
│   ├── agentaddr/
│   ├── api/
│   ├── audit/
│   ├── config/
│   ├── facts/
│   ├── index/
│   ├── observability/
│   ├── policy/
│   ├── resolver/
│   ├── revocation/
│   └── trust/

```

```
|— migrations/
|— testdata/
|   |— agents/
|   |— facts/
|   |— credentials/
|— tests/
|   |— integration/
|   |— load/
|   |— contract/
|— docker-compose.yml
|— Makefile
|— go.mod
|— README.md
```

7. Implementation Milestones

Milestone 1: Protocol primitives

Deliverables:

- AgentAddr120 encode/decode.
- Ed25519 signing and verification.
- SHA-256 digest helpers.
- AgentID normalization.
- Unit tests.

Acceptance gate:

- 100% pass on protocol primitive tests.
- Malformed records fail safely.

Milestone 2: L1 index and facts store

Deliverables:

- PostgreSQL schema/migrations.
- L1 store implementation.
- Filesystem facts store.
- Repository interfaces.
- Integration tests with Docker Compose.

Acceptance gate:

- Register and fetch L1 records.
- Store and fetch AgentFacts.
- History maintained.

Milestone 3: Registration API

Deliverables:

- /v1/agents/register.
- AgentFacts validation.

- L1 anchor creation.
- Audit event writing.

Acceptance gate:

- Register sample agents from test fixtures.
- Audit event exists for registration.

Milestone 4: Resolver API

Deliverables:

- /v1/resolve.
- L1 lookup.
- Signature validation.
- Facts fetch.
- Facts digest verification.
- Trust verification.
- Endpoint selection.
- Proof bundle response.

Acceptance gate:

- Valid agent resolves.
- Invalid trust paths fail.
- Audit event exists for allow and deny.

Milestone 5: Revocation

Deliverables:

- Credential status store.
- Revoke endpoint.
- Resolver revocation checks.
- Freshness policy.

Acceptance gate:

- Revoked credential fails resolution.
- Test verifies failure within configured window.

Milestone 6: Observability and hardening

Deliverables:

- /metrics.
- structured logs.
- trace IDs.
- health checks.
- audit hash-chain verification.
- load tests.

- chaos tests.

Acceptance gate:

- Metrics emitted for successful and failed resolution.
- Audit chain validates.
- Load test meets v0 SLOs.

8. Testing Plan

8.1 Unit Tests

- AgentAddr120 length must equal 120 bytes.
- Decode rejects records shorter or longer than 120 bytes.
- Decode rejects unsupported version.
- Decode rejects zero TTL.
- Signature verification succeeds for valid record.
- Signature verification fails after one byte is modified.
- AgentID normalization is deterministic.
- Facts digest is deterministic.
- Endpoint binding hash validates.
- Routing policy prefers adaptive > rotating > static.
- Expired endpoints are rejected.
- Missing required capability fails.
- Untrusted issuer fails.
- Revoked credential fails.

8.2 Integration Tests

- Register valid agent, then resolve successfully.
- Register agent with untrusted issuer, resolution denied.
- Register agent, corrupt facts file, resolution denied.
- Register agent, corrupt L1 record, resolution denied.
- Register agent, revoke credential, resolution denied.
- Register agent with static, rotating, and adaptive endpoints; adaptive selected.
- Stop facts store, resolution fails with clear error.
- Stop Postgres, readiness check fails.
- Verify audit event exists for each operation.
- Verify audit hash chain.

8.3 Contract Tests

- OpenAPI schema validates.
- SDK request/response types match API.
- Error response format is stable.
- Proof bundle schema is stable.

8.4 Security Tests

- Replay older sequence number rejected.
- Invalid issuer rejected.
- Expired credential rejected.
- Revoked credential rejected.
- Endpoint not present in signed facts rejected.
- Tampered AgentFacts rejected.
- Tampered L1 signature rejected.
- Audit log tampering detected by hash-chain verification.

8.5 Privacy Tests

- Direct mode may log requester metadata according to local policy.
- Private mode must not log raw requester ID.
- Private mode audit uses pseudonymous requester hash.
- Resolution response does not leak unnecessary credential chain details unless explicitly requested.

8.6 Load Tests

- Use k6 or equivalent.
- 1,000 registered agents.
- 100,000 registered agents if local resources allow.
- 95% read / 5% write.
- Resolve hot-cache path.
- Resolve cold-cache path.
- Resolve revoked credential path.

8.7 Chaos Tests

- Redis unavailable: resolver falls back to Postgres.
- Facts store unavailable: resolution fails closed.
- Revocation store unavailable: high-risk resolution fails closed.
- Audit store unavailable: registration fails; resolution behavior configurable but should emit critical error.
- Clock skew introduced: expired credentials fail conservatively.

8.6.1 v0 SLOs

Operation	Target
Hot L1 lookup	p99 < 10 ms
Full direct resolution	p99 < 300 ms
Failed trust decision	p99 < 300 ms
Audit write success rate	100% during normal operation

9. Definition of Done for v0

19. A sample agent can be registered and resolved end-to-end.
20. Resolver returns selected endpoint and proof bundle.
21. Invalid signature, invalid facts digest, untrusted issuer, missing capability, expired credential, and revoked credential are all rejected.

22. Every successful and failed path emits audit events.
23. Audit hash chain can be verified.
24. Metrics and structured logs are available.
25. Docker Compose brings up the full local system.
26. Test suite runs with one command.
27. README explains architecture, APIs, and local usage.

10. Recommended First Codex Prompt

Build the NANDA v0 repository in Go using the implementation PRD. Start with protocol primitives and the local server skeleton. Do not implement full W3C VC interoperability, OHTTP, Envoy, IPFS, Tor, or Trillian yet. Use interfaces so those can be added later.

Create:

- Go module.
- Repository layout.
- AgentAddr120 encode/decode/sign/verify package.
- AgentID normalization helper.
- SHA-256 digest helper.
- Minimal HTTP server with /healthz and /readyz.
- Unit tests for protocol primitives.
- Docker Compose skeleton with Postgres.
- Makefile with make test and make run.

Acceptance criteria for this first task:

- go test ./... passes.
- AgentAddr120 is exactly 120 bytes.
- Tampered records fail signature verification.
- Server starts locally.